LLARS: An End-to-End Pipeline for Collaborative Prompting, Batch Generation, and Hybrid Evaluation

Philipp Steigerwald[1], **Mara Stieler**2, **Jennifer Burghardt**2, **Eric Rudolph**[1], **Jens Albrecht**[1]
Technische Hochschule Nürnberg Georg Simon Ohm, Nürnberg, Germany
[1]Faculty of Computer Science, Center for Artificial Intelligence (KIZ)
[2]Faculty of Social Sciences, Institute for E-Counselling
{philipp.steigerwald, mara.stieler, jennifer.burghardt, eric.rudolph, jens.albrecht}@th-nuernberg.de

Abstract

We demonstrate LLARS, an open-source platform that bridges the gap between domain experts and developers for systematic evaluation of LLM outputs. The platform provides three integrated modules that can be used together or independently: **Collaborative Prompt Engineering** lets domain experts and developers co-author prompts in a shared real-time workspace with version control and instant LLM testing. **Batch Generation** systematically produces outputs for all combinations of prompts, LLMs, and data items with built-in cost tracking. **Large-Scale Evaluation** supports human and LLM evaluators with live agreement metrics—applicable to outputs from any source. When combined, these modules form a complete pipeline from prompt development through rigorous assessment.

Demo: https://llars-project.github.io/demo

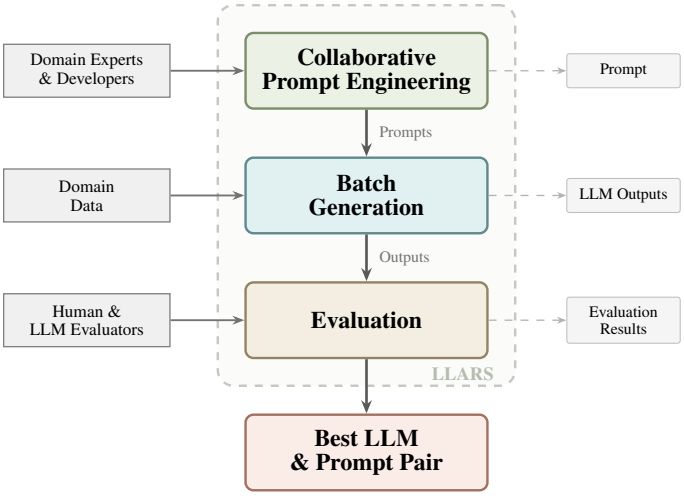

Figure 1: LLARS pipeline: domain experts and developers collaboratively develop prompts, generate outputs across LLMs, and evaluate results with human and LLM evaluators. Each stage supports export; the pipeline yields a validated model–prompt combination.

1 Introduction

Organizations increasingly build LLM-based systems for sensitive domains such as legal analysis, medical documentation, and psychosocial counseling. Developing these systems requires writing effective prompts, selecting suitable models, and rigorously evaluating outputs—tasks that demand both domain expertise and technical skill. Domain experts understand *what* constitutes quality in their field, while developers know *how* to structure prompts and configure LLM pipelines. Yet in practice, prompts are revised through multiple rounds of feedback in shared documents without version control or a common testing environment, leaving teams unsure which version was tested against which model. At the same time, organizations need to determine the right model for their use case—comparing open-source alternatives for privacy-preserving on-premise deployment or finding the most cost-effective proprietary option—but lack tools for systematic, collaborative comparison. Existing platforms address only fragments of this workflow: annotation tools like Label Studio [HumanSignal, Inc., 2024] and Argilla [Argilla, Inc., 2024] provide evaluation interfaces but no collaborative prompt development; LLM-as-Judge frameworks like MT-Bench [Zheng *et al.*, 2024] automate assessment but exclude human evaluators. No existing tool offers a unified pipeline from collaborative prompt engineering through systematic output generation to hybrid human–LLM evaluation.

We present LLARS (LLM Assisted Rating System), an open-source platform that closes this gap with an integrated three-stage pipeline (Figure 1). In the first stage, domain experts and developers *collaboratively engineer prompts* in a shared real-time editor with built-in version control, testing candidate prompts against live LLMs without leaving the workspace. Prompts can be exported for external use or passed directly into the second stage, where *batch generation* produces outputs for all combinations of data items, prompts, and models with built-in cost estimation. In the third stage, both human and LLM evaluators *assess the outputs* within the same framework—rating, ranking, comparing, or labeling—while real-time agreement metrics reveal where human and machine judgment converge or diverge.

For example, a legal team building an information extraction system can use the collaborative editor to refine prompts

with legal experts, then run batch generation across candidate models—from large proprietary to small open-source alternatives—over hundreds of documents, and finally have legal experts and LLM evaluators jointly assess whether each model reliably identifies relevant clauses rather than hallucinating when information is absent. This end-to-end workflow directly informs which model–prompt combination to deploy.

Each stage also operates independently, and results are exportable as JSON or CSV at every point. The platform connects to any OpenAI-compatible LLM provider—including locally hosted models via Ollama or vLLM—and is fully open-source.

2 Platform Overview

LLARS is a containerized web application built with Vue.js (frontend), Flask (backend), and MariaDB (storage). Real-time collaboration is provided by a dedicated Yjs [Jahns, 2023] server, and LLM access follows the OpenAI-compatible API structure, allowing teams to connect commercial providers (OpenAI, Anthropic, Google) as well as locally hosted models via Ollama. All interactions—prompt editing, generation progress, and evaluation updates—are synchronized via WebSocket.

The platform provides three modules that can be used independently or combined into a pipeline. Prompts created in the collaborative editor feed into batch generation, which produces outputs that are then assessed in the evaluation module. At each stage, results can be exported as JSON or CSV for external use.

2.1 Collaborative Prompt Engineering

Developing effective prompts for domain-specific LLM systems requires iterative refinement by those who understand the domain and those who understand prompt design. The prompt engineering module (Figure 2) addresses this through a real-time collaborative editor built on Yjs [Jahns, 2023], a CRDT framework that synchronizes edits at character granularity, merging concurrent changes without conflicts.

Users compose prompts as reorderable message blocks (e.g., system instructions, user templates) and define template variables using `{{variable}}` syntax. Variables are automatically extracted and displayed in a shared palette; during testing and batch generation, they are substituted with domain-specific data. Any collaborator can test the current prompt independently: clicking *Test* captures an atomic snapshot, fills variables with sample data, sends the prompt to a user-selected LLM, and streams the response—without interrupting concurrent editing. Multiple users can test against different models or data in parallel, enabling rapid exploration of prompt behavior across configurations.

A git-inspired version control system records each commit with multi-author attribution, tracking per-user contributions at character level. Teams can compare versions, inspect individual contributions, and roll back to previous states. Finalized prompts are exported as JSON for integration into production systems or passed directly to the batch generation module.

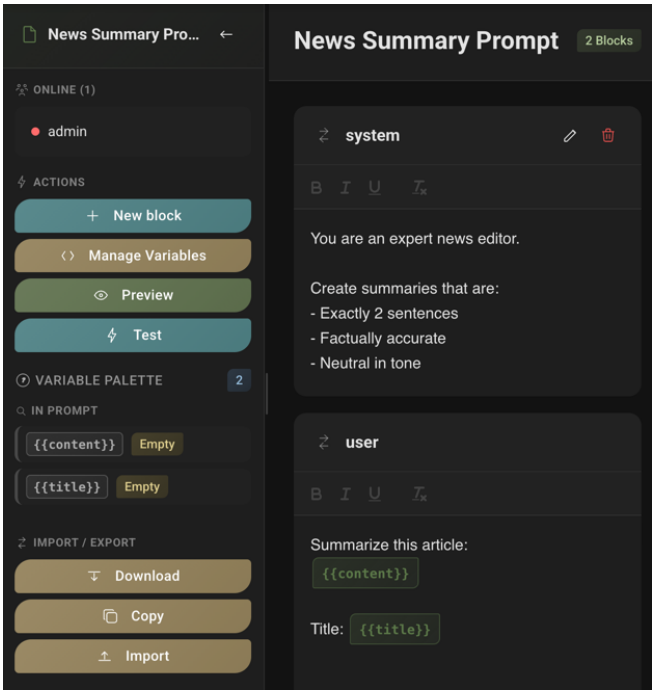

Figure 2: Collaborative prompt editor showing a news summarization prompt with two message blocks, template variables in the shared palette (left), and online presence. The sidebar provides testing, variable management, and JSON import/export.

2.2 Batch Generation

With prompts refined, the batch generation module scales output creation across combinations of data, prompts, and models (Figure 3). A guided wizard walks users through source selection, prompt and model configuration, and parameter tuning including budget limits. Sources can be existing evaluation scenarios, uploaded CSV or JSON files, or—for self-contained prompts that require no input data, such as creative text generation or instruction-following benchmarks—no source at all. The system generates the full Cartesian product: N data items $\times$ M prompts $\times$ K models. Template variables in prompts are automatically matched to data columns by name—a CSV column `content` fills every `{{content}}` placeholder across all selected prompts.

Users can register custom LLM providers with their own API keys; these models then appear throughout the platform—in prompt testing, batch generation, and as LLM evaluators. Before execution, LLARS estimates costs based on model pricing and token projections; optional budget caps automatically pause jobs that exceed thresholds. All generation progress—including token-level output streaming—is broadcast via WebSocket, allowing team members to join and monitor running jobs in real time. Each completed output retains full provenance—source item, prompt template, model, token count, and cost—exportable as CSV or JSON. Completed batches can be transferred directly into the evaluation module through the integrated Scenario Wizard, preserving model attribution for systematic cross-model comparison.

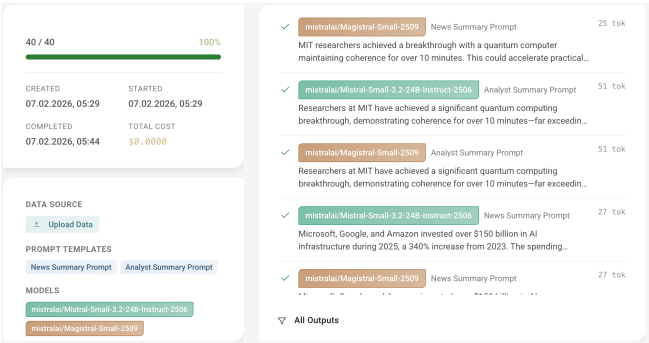

Figure 3: Batch generation results for a news summarization job: two prompts tested across two LLMs on ten articles, producing 40 outputs with full provenance. The *Scenario Wizard* button transfers outputs directly to evaluation.

2.3 Evaluation

The evaluation module enables human evaluators and LLM evaluators to assess outputs within a unified framework, supporting four general evaluation types (Figure 4): *rating* with configurable multi-dimensional Likert scales and weighted dimensions (e.g., coherence, fluency, relevance), *ranking* where evaluators sort items into ordered buckets, *labeling* for categorical classification (binary or multi-class), and pairwise *comparison*. The framework is extensible with domain-specific types; LLARS includes two for psychosocial counseling research (Section 3).

An AI-assisted Scenario Wizard guides scenario creation through five steps—data upload, task selection, configuration, team assembly, and review. The wizard analyzes uploaded data structures using deterministic pattern matching with an LLM fallback for ambiguous cases, and recommends the most suitable evaluation type along with matching presets. Scenarios can also be created directly from batch generation outputs, preserving model attribution so that evaluators compare outputs across LLMs for the same input.

Scenario owners control how items are distributed: all items to every evaluator for full inter-rater analysis, or configurable subsets for faster throughput. Both human evaluators and LLM evaluators participate in the same scenarios; LLM evaluators use type-specific prompts that administrators can customize per scenario type. As evaluations are submitted, LLARS computes agreement metrics in real time—Krippendorff's α, Cohen's κ, Fleiss' κ, Kendall's τ and W, Spearman's ρ, and intraclass correlation (ICC), among others—and scenario owners can filter by evaluator type (human–human, LLM–LLM, or human–LLM) to analyze where machine and human judgment diverge. All evaluation data, metrics, and configurations can be exported as JSON or CSV for further analysis.

3 Application: Online Counselling

LLARS is deployed in ongoing research on CAIA [Steigerwald *et al.*, 2025], an AI-assisted support system for text-based psychosocial counselling with seven LLM-driven functionalities—from case summarisation to hypothesis generation. Counselling researchers and developers used the collaborative prompt editor to jointly develop the prompts underlying these functionalities, testing iteratively against counselling data until outputs met professional standards. For one of these functionalities—automated subject line generation [Steigerwald and Albrecht, 2025]—the complete pipeline was exercised: batch generation produced 253 subject lines across 11 LLMs at different quantisation levels, and five counselling professionals together with an AI evaluator rated the outputs, yielding 1,518 assessments. LLARS's cross-model comparison directly informed the selection of a privacy-preserving open-source model for local deployment.

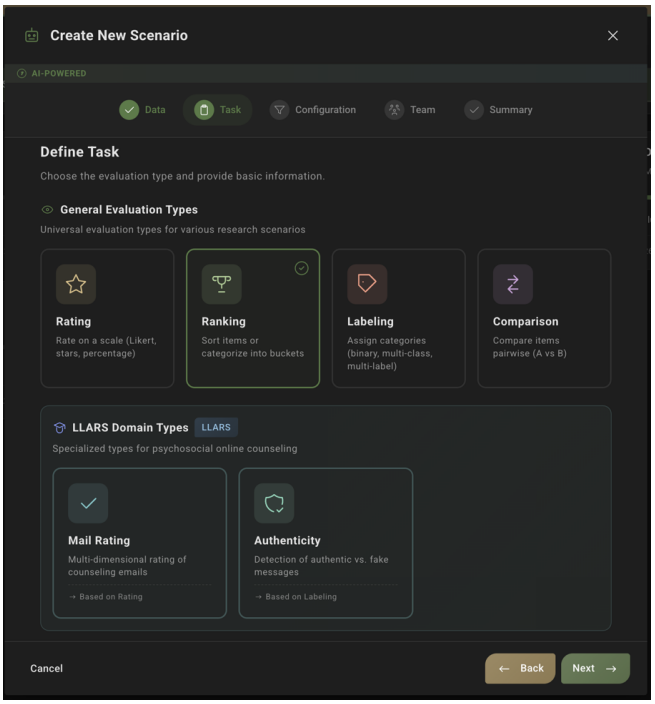

Figure 4: AI-assisted Scenario Wizard showing the task selection step with four general evaluation types and two domain-specific types for psychosocial counseling. The five-step wizard analyzes uploaded data and recommends the most suitable evaluation type.

These projects also motivated two domain-specific evaluation types (Figure 4): *Mail Rating* for assessing AI-generated counsellor responses on dimensions such as coherence and quality, and *Authenticity* (Figure 5) for classifying counselling threads as real or AI-generated, with ground truth labels enabling classification accuracy alongside inter-rater agreement.

4 Related Work

4.1 GUI-First Tooling for Prompt Engineering and Evaluation

Recent years have seen rapid growth in GUI-first tooling around large language models (LLMs), particularly for prompt iteration, dataset-based experiments, and automated evaluation. In practice, many teams—especially in high-stakes domains—require workflows that support (i) interactive prompt engineering with rapid feedback, (ii) systematic

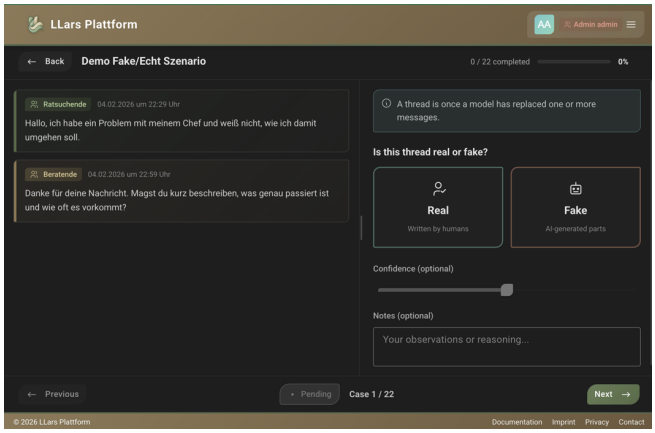

Figure 5: Authenticity evaluation: evaluators read a counselling conversation and classify it as written by humans or containing AI-generated parts, with optional confidence and reasoning.

batch generation across multiple prompts, models, and domain datasets, and (iii) rigorous output evaluation involving both human raters and automated (often LLM-based) judges. Our focus is specifically on tools that provide end-user-facing GUIs and are suitable for mixed teams of domain experts and developers.

4.2 Prompt Engineering Interfaces and Dataset-Based Experimentation

Several systems provide prompt playgrounds and UI-driven experimentation over datasets. Phoenix includes a Prompt Playground and supports running prompts over collections of inputs while tracking results in an experiment-oriented workflow (https://arize.com/docs/phoenix/prompt-engineering/overview-prompts/prompt-playground; https://arize.com/docs/phoenix/prompt-engineering/how-to-prompts/using-the-playground). Langfuse offers prompt management and UI-based experiments on datasets, enabling structured comparisons across prompt variants (https://langfuse.com/docs/prompt-management/overview; https://langfuse.com/docs/evaluation/experiments/experiments-via-ui). LangSmith provides a prompt playground tightly integrated with dataset-level evaluation runs (https://docs.langchain.com/langsmith/run-evaluation-from-prompt-playground; https://docs.langchain.com/langsmith/evaluation-quickstart). Weave similarly exposes an evaluation playground that can execute runs on datasets and compare outcomes (https://docs.wandb.ai/weave/guides/tools/evaluation_playground). Finally, ChainForge provides a visual environment for exploring prompt/model combinations and analysing LLM outputs (https://www.chainforge.ai/docs/; https://github.com/ianarawjo/ChainForge).

These interfaces substantially reduce iteration friction compared to code-only approaches. However, for many tools the "prompt-to-experiment" experience is primarily designed around developer workflows (e.g., tracing/observability, app iteration), and the degree to which they support structured, domain-expert-facing co-authoring and versioning varies widely. This is reflected in Table 1 by partial marks (~) in the "Process chain" and/or "Prompt" dimensions where collaboration, provenance, or evaluation handoff is not a first-class, integrated GUI stage.

4.3 Evaluation: Automated Judges, Human Assessment, and Hybrid Setups

A key trend is the integration of automated evaluation, including "LLM-as-a-judge" patterns. Phoenix documents LLM-as-a-judge evaluators as part of its evaluation stack (https://arize.com/docs/phoenix/evaluation/concepts-evals/llm-as-a-judge). Langfuse also provides LLM-as-a-judge methods (https://langfuse.com/docs/evaluation/evaluation-methods/llm-as-a-judge), and Weave supports judge-driven scoring inside its evaluation UI (https://docs.wandb.ai/weave/guides/tools/evaluation_playground). Commercial platforms such as Braintrust and Maxim provide evaluation/scoring and experiment reporting with a strong focus on production feedback loops (https://www.braintrust.dev/docs/evaluate; https://www.getmaxim.ai/docs/offline-evals/via-ui/prompts/prompt-evals). Vellum similarly offers prompt engineering and an evaluations product for systematic comparisons (https://www.vellum.ai/products/prompt-engineering; https://www.vellum.ai/blog/introducing-vellum-evaluations).

In contrast, domain settings that require careful quality assurance often still rely on humans as the gold standard. Label Studio has introduced dedicated LLM evaluation templates and workflows for comparing model outputs (including scenarios without ground truth) (https://labelstud.io/blog/how-to-evaluate-and-compare-llms-using-prompts-in-label-studio/; https://labelstud.io/blog/new-llm-evaluation-templates-for-label-studio/), but it is typically used by importing externally generated outputs as labeling tasks. Argilla emphasises collaboration and feedback collection for LLM workflows (https://argilla.io/blog/argilla-for-llms/; https://github.com/argilla-io/argilla), yet it is not positioned as a unified prompt→batch→hybrid-evaluation pipeline for systematic cross-product studies. Consequently, multiple tools receive partial marks (~) in the "Eval (H+LLM)" dimension when their primary workflow is judge-centric or when human evaluation is supported but not as a structured, first-class campaign workflow within the same process chain.

4.4 Integrated Process Chains and the Closest Competitor

The strongest overlap with our target pipeline is found in Agenta, an open-source platform that explicitly combines prompt development, evaluation, and human annotation in one product (https://agenta.ai/docs/; https://github.com/Agenta-AI/agenta). This makes Agenta an important reference point when discussing "unified pipeline" claims. Nevertheless, we mark Agenta as partial (~) for "Process chain like LLARS" and "Batch (P×M×D)" because its primary framing is LLMOps and application lifecycle management rather than evaluation-centric, matrix-style experimentation in which the cartesian product of prompts×models×data is treated as the default experimental unit and paired with ded-

icated preference-style human assessment (ratings/rankings) and agreement analytics.

4.5 Why LLARS is Needed

LLARS is designed around a pipeline-first interaction model for bridging domain experts and developers in comparative studies (https://llars-project.github.io/demo). Specifically, LLARS couples (i) collaborative prompt engineering with live testing, (ii) systematic batch generation over prompts×models×data with explicit provenance and budgeting as first-class concepts, and (iii) preference-style evaluation (ratings/rankings/pairwise comparison) that supports both human raters and LLM-based evaluators within the same workflow. This focus is complementary to broader LLMOps platforms: while many tools prioritise observability, deployment iteration, or judge-driven metrics, LLARS centres the structured assessment process required in domains where human evaluation remains essential but costly. Table 1 summarises how existing GUI-first tools cover parts of this chain, and clarifies that LLARS' contribution is the tightly integrated process chain optimised for domain-expert-centred evaluation.

5 Demonstration and Conclusion

Our live demonstration walks attendees through the complete pipeline: collaboratively editing a prompt, generating outputs across multiple LLMs with real-time cost tracking, and evaluating results with both human and LLM evaluators while observing agreement metrics update live. Attendees can bring their own data or use provided examples. LLARS is open-source and runs via Docker (16 GB RAM, 4+ CPU cores recommended).

By integrating collaborative prompt engineering, systematic batch generation, and hybrid human–LLM evaluation into a single GUI-driven platform, LLARS addresses a gap in existing tooling: no prior system offers domain experts and developers a shared environment spanning the full cycle from prompt development to rigorous assessment. Ongoing deployments in psychosocial online counselling demonstrate that this integrated pipeline supports real-world research workflows. Future work includes a formal user study and extending the platform with fine-tuning integration and automated prompt optimization.

References

[Argilla, Inc., 2024] Argilla, Inc. Argilla: Collaboration tool for AI engineers and domain experts. https://argilla.io, 2024. Accessed: 2026-01-06.

[HumanSignal, Inc., 2024] HumanSignal, Inc. Label studio: Open source data labeling platform. https://labelstud.io, 2024. Accessed: 2026-01-06.

[Jahns, 2023] Kevin Jahns. Yjs: A CRDT framework for shared editing. https://yjs.dev, 2023. Accessed: 2026-01-06.

[Steigerwald and Albrecht, 2025] Philipp Steigerwald and Jens Albrecht. Comparing large language models for automated subject line generation in e-mental health: A performance study. In *Proceedings of the 11th International Conference on Information and Communication Technologies for Ageing Well and e-Health (ICT4AWE)*, pages 70–77. SciTePress, 2025.

[Steigerwald *et al.*, 2025] Philipp Steigerwald, Nico Bienlein, Jennifer Burghardt, Mara Stieler, Robert Lehmann, and Jens Albrecht. CAIA in practice: Field evaluation of an AI-assisted support system for text-based online counselling. In *Proceedings of the IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, 2025. Accepted.

[Zheng *et al.*, 2024] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

Table 1: Comparison of GUI-first tools for prompt engineering, batch generation, and hybrid LLM output evaluation. Legend: ✓ = native first-class support in the GUI; ˜ = partial support (limited scope, requires workarounds/external steps, or not a first-class GUI workflow); – = not supported / not a primary feature. "Process chain like LLARS" denotes a single-GUI workflow from prompt live-testing → batch runs over prompts×models×data → evaluation involving human raters and LLM/judge components.

Tool	Open	Process	Prompt	Batch	Eval (H+LLM)	Primary URLs
LLARS	✓	✓	✓	✓	✓	https://llars-project.github.io/demo
Agenta	✓	˜	✓	˜	✓	https://agenta.ai/docs/; https://github.com/Agenta-AI/agenta
Arize Phoenix	✓	˜	✓	˜	˜	https://arize.com/docs/phoenix/prompt-engineering/overview-prompts/prompt-playground; https://arize.com/docs/phoenix/prompt-engineering/how-to-prompts/using-the-playground; https://arize.com/docs/phoenix/evaluation/concepts-evals/llm-as-a-judge; https://github.com/Arize-ai/phoenix
Langfuse	✓	˜	˜	✓	˜	https://langfuse.com/docs/prompt-management/overview; https://langfuse.com/docs/evaluation/experiments/experiments-via-ui; https://langfuse.com/docs/evaluation/evaluation-methods/llm-as-a-judge; https://github.com/langfuse/langfuse
LangSmith	–	˜	✓	✓	˜	https://docs.langchain.com/langsmith/run-evaluation-from-prompt-playground; https://docs.langchain.com/langsmith/evaluation-quickstart
W&B Weave	✓	˜	✓	✓	˜	https://docs.wandb.ai/weave/guides/tools/playground; https://docs.wandb.ai/weave/guides/tools/evaluation_playground; https://github.com/wandb/weave
ChainForge	✓	˜	✓	✓	˜	https://www.chainforge.ai/docs/; https://github.com/ianarawjo/ChainForge
Label Studio	✓	˜	˜	˜	˜	https://labelstud.io/blog/how-to-evaluate-and-compare-llms-using-prompts-in https://labelstud.io/blog/new-llm-evaluation-templates-for-label-studio/; https://github.com/HumanSignal/label-studio
Argilla	✓	–	–	–	˜	https://argilla.io/blog/argilla-for-llms/; https://github.com/argilla-io/argilla
Braintrust	–	˜	˜	˜	˜	https://www.braintrust.dev/docs/evaluate; https://www.braintrust.dev/docs/best-practices/scorers
Maxim AI	–	˜	✓	✓	˜	https://www.getmaxim.ai/docs/offline-evals/via-ui/prompts/quickstart; https://www.getmaxim.ai/docs/offline-evals/via-ui/prompts/prompt-evals
Vellum	–	˜	✓	˜	˜	https://www.vellum.ai/products/prompt-engineering; https://www.vellum.ai/blog/introducing-vellum-evaluations